

Skip List

–Display List

–Time Complexity Analysis

Program:

```
// Display the skip list layer by layer
void displayList(struct SkipList *lst) {
    cout << "\n--- Skip List ---" << endl;
    for (int i = 0; i <= lst->level; i++) {
        struct Node *node = lst->header->forward[i];
        cout << "Level " << i << ": ";
        while (node != NULL) {
            cout << node->key << " ";
            node = node->forward[i];
        }
        cout << endl;
    }
    cout << "-----" << endl;
}

case 4:
    displayList(list);
    break;
```

Time Complexity Analysis

1. Best Case Time Complexity: $O(n)$ or $O(1)$

- **Populated List ($O(n)$):** Even if the Skip List is an "All Tails" completely flat list (maximum level is 0), the `displayList` function still has to print every single element. The outer loop runs once ($i = 0$), and the inner loop traverses all n nodes on the ground floor. Therefore, if there are n elements to print, the absolute fastest it can run is linear $O(n)$ time.
- **Empty List ($O(1)$):** The only true constant – time scenario occurs if the Skip List contains exactly zero elements. The outer loop initializes, sees that the `header → forward[0]` is NULL, skips the inner `while` loop completely, and immediately exits. This evaluates in instantaneous $O(1)$ time.

2. Worst Case Time Complexity: $O(n \log n)$ or $O(n)$

The worst – case scenario happens if the insertion phase suffered a catastrophic All Heads statistical anomaly, promoting every single node to the absolute maximum level.

- **Theoretical Worst Case ($O(n \log n)$):** If the list has no hard cap [fixed constant] and the maximum level is allowed to scale to $\log n$, then all n nodes will exist on all $\log n$ levels.

The `displayList` function will be forced to print n nodes horizontally across $\log n$ separate floors.

This results in $O(n \log n)$ time.

- **Practical Worst Case ($O(n)$):** If the program enforces a strict, constant ceiling (e.g., `MAX_LEVEL = 3`), the outer loop can only ever run a maximum of 4 times. Even if all n nodes are clogged on all 4 floors, the loop runs $4n$ times. Because 4 is a constant, it drops away, leaving a practical worst – case complexity of $O(n)$.

Average Time Complexity

1. The Execution Mechanics

The **displayList** algorithm consists of two nested loops:

- **Vertical Traversal:** The outer **for** loop dictates the current floor, starting at **i = 0** and climbing to **lst → level**.
- **Horizontal Traversal:** The inner **while** loop scans across that specific floor, printing **node → key** until it hits **NULL**.

Instead of multiplying the maximum height by the maximum width (which would incorrectly imply an **$O(n \log n)$** average), the algorithm simply touches every single valid **forward** pointer exactly once to print its value. Therefore, the total execution time is perfectly equal to the total number of pointers in the entire structure.

2. The Expected Number of Pointers

In a well – balanced Skip List (using a fair coin toss where **p = 0.5**), the probabilistic balancing ensures that the number of nodes halves on every subsequent floor.

In the specific **randomLevel()** code used in this program, Heads(a successful coin flip) corresponds to 0.

Here is exactly how the program simulates the coin toss using the modulo operator:

- **Heads (Success = 0):** The program evaluates **(rand()% 2) == 0**. If the random number generator produces an even number, the result is 0. The program treats this as a successful “Heads,” increases the node's level (**lvl + +**), and loops to flip the coin again.
- **Tails (Failure = 1):** If the random number generator produces an odd number, the result of **rand()% 2** is 1. The program treats this as a failed Tails, the **while** loop condition instantly becomes False, and the node stops growing.

- **Level 0:** Contains all n nodes.

Level 0 (The Ground Floor): Every single element inserted into the list must have a pointer on the ground floor. There is a **100%** probability of reaching Level 0, meaning all n elements exist here.

- **Level 1:** Contains an expected $\frac{n}{2}$ nodes.

Level 1: To grow to Level 1, each of those n nodes flips a coin. Because the chance of getting Heads is exactly **50%**, we mathematically expect only half of the elements from **Level 0** to be promoted. Therefore, the total expected pointers on this floor is $\frac{n}{2}$.

- **Level 2:** Contains an expected $\frac{n}{4}$ nodes.

Level 2: Now, only those $\frac{n}{2}$ nodes that made it to Level 1 get to flip the coin again. They have a **50%** chance to win a second time. Only half of **Level 1** is expected to be promoted. Half of $\frac{n}{2}$ gives us $\frac{n}{4}$ expected pointers.

- **Level 3:** Contains an expected $\frac{n}{8}$ nodes.

Level 3: The surviving $\frac{n}{4}$ nodes flip the coin a third time. Again, we expect half of them to win and half to lose. Half of $\frac{n}{4}$ gives us $\frac{n}{8}$ expected pointers.

To find the total work the algorithm must perform, we sum the nodes across all levels. This forms an infinite geometric series:

$$Total = n + \frac{n}{2} + \frac{n}{4} + \frac{n}{8} + \dots$$

factor out the n:

$$TotalSpace = n \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots \right)$$

We know ,through geometric infinite series : $s_{\infty} = \frac{a_1}{1-r}$

$$a_1 = 1 \text{ (First Term) and } r = \frac{\text{Any Term}}{\text{Previous Term}} \Rightarrow \frac{1}{2} = \frac{1}{2} ; \frac{1/4}{1/2} = \frac{1}{2} \text{ etc.}$$

$$\therefore S_{\infty} = \frac{a_1}{1-r} = \frac{1}{1-\frac{1}{2}} = \frac{1}{\frac{1}{2}} = 2.$$

$$TotalSpace = n \times S_{\infty} = n \times 2 = 2n.$$

Alternatively ,Algebraic Shifting Method:

$$\text{Let , } S = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now, Multiplying $\frac{1}{2}$ on both the sides we get:

$$\Rightarrow \frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$

Now subtracting we get:

$$\Rightarrow S - \frac{1}{2}S = (1 - 0) + \left(\frac{1}{2} - \frac{1}{2}\right) + \left(\frac{1}{4} - \frac{1}{4}\right) + \left(\frac{1}{8} - \frac{1}{8}\right) + \dots$$

$$\Rightarrow S - \frac{1}{2}S = 1$$

$$\Rightarrow S \left(1 - \frac{1}{2}\right) = 1$$

$$\Rightarrow S \left(\frac{1}{2}\right) = 1$$

$$\Rightarrow S = 2$$

$$\text{TotalSpace} = n \times S = n \times 2 = 2n.$$

*Because the nested loops must execute exactly once for every valid pointer, they will perform an average of **2n** operations to print the entire list. In Big – O notation, constant multipliers are discarded to focus on the scaling trend.*

*Leaving out constant : **2** , leaves a strict average time complexity of **O(n)**.*

Hence, Average Time Complexity : O(n)
